

Jinwon Lee

Application Software · AI Platform · End-to-End Delivery

Website

<https://jinwonlee.github.io/>

This portfolio summarizes end-to-end delivery of enterprise and user-facing applications from requirements and architecture through implementation, rollout, and production stabilization. Each project is organized around the problem context, my role, engineering and delivery focus, outcomes, and capability signals.

Application Software

Enterprise and user-facing apps, asynchronous architecture, API and system integration

End-to-End Delivery

Requirements, architecture, implementation, rollout readiness, and production stabilization

Reliability

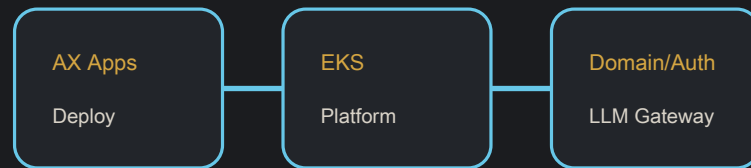
Authentication, authorization, observability, incident response, and system performance

Cloud Platform

Automation with Azure, AWS, OpenShift, Kubernetes, GitOps, and Infrastructure as Code

Project Details

Problem, role, engineering/delivery focus, outcome, and capability point

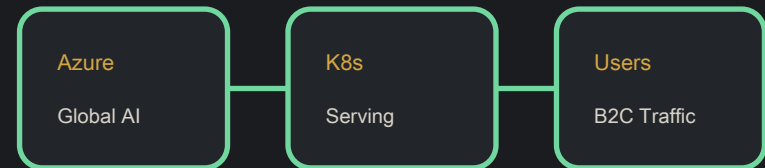


01. Internal AX Agent Platform

AWS EKS-based platform for deploying and operating AX applications with automated domains, access control, governance, and LLM Gateway integration.

- Problem** Enable teams to deploy AX applications quickly while enforcing enterprise access control, security, and AI governance.
- Role** Owned end-to-end platform delivery from architecture and implementation through deployment and operation, including authentication, authorization, security, and internal governance.
- Engineering/Delivery** AWS EKS, AX Agent runtime adaptation, automated domain provisioning, access control, policy enforcement, and client integration with the existing LLM Gateway.
- Outcome** Created a self-service path for governed AX application deployment with automatic domains and authorized-only access.
- Capability** Experience building a governed self-service agent platform with runtime compatibility, access control, and enterprise LLM integration.

AWS EKS Kubernetes AuthN/AuthZ Governance



02. Global B2C Generative AI Service

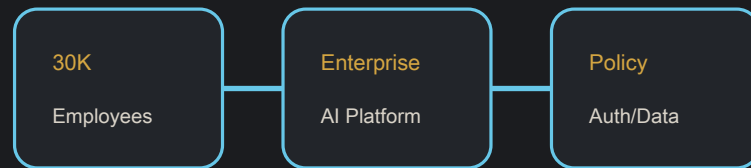
Microsoft Azure-based B2C generative AI service expanded to the UK with initial production operations and incident response.

- Problem** Expand a consumer-facing generative AI service to the UK and stabilize its initial production operations.
- Role** Delivered platform/SRE responsibilities for UK rollout readiness, initial production operations, incident response, and early stabilization.
- Engineering/Delivery** Azure and Kubernetes operations, rollout readiness, production stabilization, and incident response.
- Outcome** Delivered UK rollout readiness and early production stabilization, completing the engagement in June 2026.
- Capability** Experience delivering UK rollout readiness, production stabilization, and incident response for a consumer generative AI service.

Azure Kubernetes Generative AI SRE

Project Details

Problem, role, engineering/delivery focus, outcome, and capability point

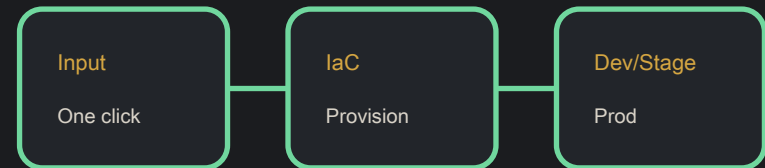


03. Company-wide AI Chatbot Platform (30K Users)

Enterprise AI chatbot platform serving 30,000 employees with Kubernetes-based application delivery, reliability, and data protection.

- Problem** Provide a reliable internal AI assistant platform for tens of thousands of employees.
- Role** Led platform and application reliability across development, staging, and production, connecting releases with security and operational requirements.
- Engineering/Delivery** Kubernetes operations, Helm/Helmfile IaC, AI service delivery, authentication, and data protection compliance.
- Outcome** Supported a company-wide AI service with production-grade operating standards.
- Capability** Experience leading application delivery and reliability for tens of thousands of internal users.

Kubernetes Internal Cluster Application Delivery ArgoCD



04. Kubernetes Cluster Automation

One-click Kubernetes cluster provisioning for repeatable developer and test environments.

- Problem** Cloud developers needed repeatable Kubernetes environments without manual setup overhead.
- Role** Designed and implemented provisioning automation for Kubernetes clusters and shared platform modules.
- Engineering/Delivery** One-click provisioning, Python automation, reusable cluster modules, and developer environment standardization.
- Outcome** Reduced manual operations and improved environment consistency for cloud development workflows.
- Capability** Experience standardizing infrastructure and automating repeatable development and test environments.

Kubernetes Terraform Ansible Python

Project Details

Problem, role, engineering/delivery focus, outcome, and capability point



05. LLM Performance Evaluation Pipeline

Automated evaluation pipeline for repeatable LLM quality and performance comparison across Machine Translation tasks.

- Problem** LLM model quality needed repeatable evaluation rather than ad hoc manual checks.
- Role** Built the evaluation workflow and metric tracking path for Machine Translation model comparison.
- Engineering/Delivery** Python evaluation automation, PyTorch-based workloads, MLflow metric tracking, and Kubernetes execution.
- Outcome** Made LLM evaluation results easier to compare and reproduce across model candidates.
- Capability** Experience automating model quality comparison and making evaluation results traceable and repeatable.

Python

PyTorch

MLflow

Evaluation Automation



06. GitOps CI/CD Pipeline with ArgoCD

GitOps-based delivery pipelines for containerized AI/platform services with Python automation and DevSecOps checks.

- Problem** Containerized AI/platform services needed repeatable, auditable, and secure deployment workflows.
- Role** Designed delivery automation and connected AI service deployment pipelines with security verification steps.
- Engineering/Delivery** GitOps, GitHub Actions, Docker image handling, Kubernetes deployment, and Python glue automation.
- Outcome** Improved deployment consistency and reduced reliance on manual release operations.
- Capability** Experience connecting deployment consistency, security checks, and operations automation for containerized services.

ArgoCD

GitHub Actions

Docker

Kubernetes

Python

Project Details

Problem, role, engineering/delivery focus, outcome, and capability point



07. Centralized Monitoring & Observability Stack

Centralized observability stack for system metrics, container runtime metrics, dashboards, and alerts.

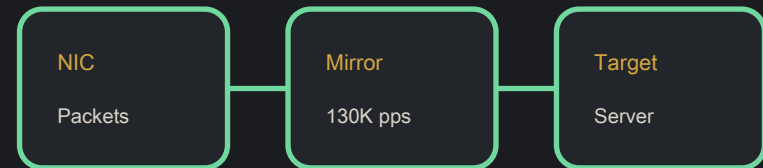
- Problem** Production services needed shared visibility into system behavior and container runtime health.
- Role** Built the monitoring path and dashboards used for platform and service operations.
- Engineering/Delivery** Prometheus metrics, Grafana dashboards, OpenTelemetry signals, Loki logging, and alert rules.
- Outcome** Improved operational visibility for incident detection and runtime analysis.
- Capability** Experience using metrics, logs, and traces for incident detection and runtime analysis.

Grafana

Prometheus

OpenTelemetry

Loki



08. High-throughput Packet Mirroring Application

Real-time packet mirroring application that forwarded up to 130,000 packets/sec under constrained resources.

- Problem** Network/application teams needed real-time packet forwarding under limited system resources.
- Role** Implemented the packet mirroring application and tuned it for resource-constrained operation.
- Engineering/Delivery** High-throughput I/O, Linux runtime behavior, resource limits, and real-time data forwarding.
- Outcome** Delivered a system component capable of forwarding up to 130,000 packets/sec.
- Capability** Experience optimizing Linux runtime behavior and high-throughput data movement under resource constraints.

Linux

Networking

System Software

Performance

Project Details

Problem, role, engineering/delivery focus, outcome, and capability point



09. Distributed CNN Training System

Research on accelerated CNN distributed deep learning with automatic resource-aware layer placement.

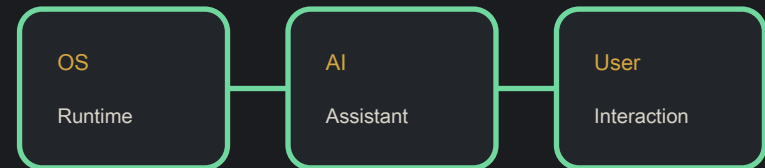
- Problem** Distributed CNN training suffered from network bottlenecks and inefficient layer placement.
- Role** Contributed to research on resource-aware placement for distributed TensorFlow/CNN training.
- Engineering/Delivery** Distributed systems, model parallelism, resource-aware scheduling, TensorFlow, and GPU-aware workloads.
- Outcome** Published research artifacts including USENIX NSDI 2019 poster and arXiv paper; reported up to 2.3x training speedup.
- Capability** Research experience on network bottlenecks, resource placement, and performance improvement for distributed training workloads.

TensorFlow

Distributed Systems

AI Workloads

Python



10. Siri-like AI Assistant on TmaxOS

AI assistant with system-level integration and natural user experience.

- Problem** The OS environment needed an AI assistant that could connect user interaction with system-level capabilities.
- Role** Built AI inference and preprocessing flows and integrated them with TmaxOS-level behavior.
- Engineering/Delivery** Async architecture, system integration, AI model inference, and data preprocessing.
- Outcome** Delivered an AI assistant experience with smoother interaction through asynchronous design.
- Capability** Experience integrating AI functionality with OS-level behavior, asynchronous processing, and user interaction.

Python

AI/ML

TmaxOS

Async Programming

Links and References

Detailed projects and the latest CV are available on the online portfolio.

Website

<https://jinwonlee.github.io/>

GitHub

<https://github.com/JINwonLEE>

LinkedIn

<https://www.linkedin.com/in/jwl1993>

Email

dlwlsdnjsehs1993@gmail.com

Publications

- Alleviating the Network Bottleneck for CNN Distributed Training through Automatic Resource-Aware Layer Placement — USENIX NSDI 2019 (2019)
- Accelerated Training for CNN Distributed Deep Learning through Automatic Resource-Aware Layer Placement — arXiv 2019 (2019)
- Improving Performance of Distributed TensorFlow using CNN Characteristics Exploiting Model Parallelism — ACM EuroSys 2018 (2018)

This document is a portfolio summary based on publicly shareable career and project information. It does not include confidential implementation details or non-public information.